

Лабораторная работа №1

Управление доступом с помощью списков контроля доступа

Части: 1.1 — управление доступом к объектам операционных систем; 1.2 — управление доступом к приложениям и системам баз данных

Вариант части 1.2: PostgreSQL — реляционная СУБД

Реализация: `acl_lab.py` (17 шагов задания), `pg_acl.py` (установка, настройка, проверка, удаление)

Окружение: одноразовый контейнер Debian, запуск от root

Результат проверок: 22 проверки политики СУБД, все PASS

1. О чём эта работа простыми словами	2
2. Часть 1.1: как устроены права в файловой системе	2
3. Четыре правила, которые всё объясняют	4
4. Что реализовано в части 1.1	6
5. Неочевидные результаты прогона	6
6. Часть 1.2: контроль доступа в СУБД	8
7. Что реализовано в части 1.2	9
8. Вопросы и ответы	11
9. Итог	13

1. О чём эта работа простыми словами

Задача работы — научиться отвечать на вопрос «кому и что разрешено» и уметь это настроить и проверить.

Любая система защиты начинается с трёх понятий:

Понятие	Что это	Пример из работы
Субъект	тот, кто хочет что-то сделать	пользователь <code>iit11</code> , группа <code>group_iit1</code> , роль <code>guest_role</code>
Объект	то, над чем действие производится	файл <code>file11</code> , каталог <code>pzs11</code> , таблица <code>public_data</code>
Право	какое именно действие разрешено	чтение, запись, выполнение, <code>SELECT</code> , <code>INSERT</code>

Список контроля доступа (ACL) — это просто таблица, которая хранится рядом с объектом и говорит: такой-то субъект может делать с этим объектом такие-то действия. Одна строка такой таблицы называется **записью ACL** (Access Control Entry, ACE).

ГЛАВНАЯ ИДЕЯ РАБОТЫ

В работе одна и та же модель разграничения доступа изучается на двух очень разных объектах: на файлах операционной системы (часть 1.1) и на таблицах базы данных (часть 1.2). Смысл в том, чтобы увидеть: механизм один и тот же — субъект, объект, право — а реализации разные, и у каждой свои неочевидные правила.

2. Часть 1.1: как устроены права в файловой системе

2.1. Три класса вместо длинного списка

Полноценный ACL мог бы содержать сколько угодно записей: «Иванову — чтение, Петрову — запись, группе бухгалтеров — всё». Но традиционная модель UNIX для экономии места хранит всего **три записи**:

Класс	Кто в него попадает
Владелец (owner, u)	один конкретный пользователь — тот, кто создал файл
Группа (group, g)	все члены группы-владельца файла
Остальные (other, o)	все прочие пользователи системы

Для каждого класса хранятся три бита: **r** — чтение, **w** — запись, **x** — выполнение. Всего девять битов, они и называются «режимом доступа» (mode).

2.2. Как читать восьмеричную запись

Права записывают числом в восьмеричной системе — по одной цифре на класс. Каждая цифра складывается из трёх весов:

$$r = 4 \cdot w = 2 \cdot x = 1$$

Режим	Буквами	Что значит
0700	rwX-----	владельцу всё, остальным ничего
0400	r-----	владельцу только чтение
0040	---r-----	только группе чтение, владельцу — ничего
0007	-----rwx	всё «остальным», а владельцу и группе — ничего
0777	rwXrwXrwX	всё всем

Обратите внимание на строки `0040` и `0007`: владелец файла может иметь **меньше** прав, чем посторонние. Это не ошибка, а прямое следствие правила, разобранный в следующем разделе.

2.3. Права на файл и права на каталог — это разные вещи

Самая частая ошибка в понимании — считать, что права каталога и права файла означают одно и то же. Для каталога те же три бита читаются совершенно иначе:

Бит	Для файла	Для каталога
r	прочитать содержимое	получить список имён внутри (<code>ls</code>)
w	изменить содержимое	создавать и удалять файлы внутри
x	запустить как программу	войти в каталог, обратиться к файлу по имени

СЛЕДСТВИЕ, КОТОРОЕ УДИВЛЯЕТ

Право удалить файл определяется правами на **каталог**, а не на сам файл. Если у вас есть **w** на каталог, вы удалите оттуда даже файл с режимом `0000`, принадлежащий другому пользователю. И наоборот: имея все права на файл, но не имея **w** на каталог, удалить его невозможно.

В прогоне это видно прямо: в столбце «удаление» стоит «25 из 25» у всех, кто имеет **w** на каталог, независимо от режимов самих файлов.

3. Четыре правила, которые всё объясняют

Почти все «странные» результаты прогона объясняются этими четырьмя правилами. Их стоит знать формулировками.

Правило 1. Решение принимает ровно одна запись ACL

Проверка идёт в строгом порядке: **владелец** → **группа** → **остальные**. Как только система определила, к какому классу относится субъект, она смотрит **только этот класс** и на остальные уже не оглядывается.

Поэтому если вы владелец файла с режимом `0040`, вы попадаете в класс «владелец» с правами `---` и получаете отказ. То, что классу «группа» разрешено чтение, вам не поможет — вы в этот класс не попали.

КАК ЭТО ПРОЯВИЛОСЬ В РАБОТЕ

Файлы `file21` – `file25` по заданию должны иметь права только для группы, а создаёт их пользователь `iit11`, который сам входит в эту группу. Результат: `iit11` — владелец, значит попадает в класс владельца с нулевыми правами и не может открыть собственный файл. Это корректное поведение модели POSIX, и его нужно назвать именно так, а не «ошибкой программы».

По этой же причине владельцем каталогов `pzs12` и `pzs13` в программе намеренно назначен `root`: иначе `iit11` оказался бы владельцем и потерял бы доступ, а шаг 13 задания (создание файлов от имени `iit11`) стал бы невыполнимым.

Правило 2. Чтобы запустить сценарий, нужны сразу два бита

Скомпилированную программу достаточно иметь право выполнять (`x`). Но файл со сценарием сначала должен прочитать интерпретатор — а он работает от имени запускающего пользователя. Значит нужны **и x, и r**.

Файлы `filex5` по заданию имеют только бит выполнения (`0100`, `0010` и так далее). Поэтому запустить их не может никто — не хватает бита чтения. В программе это решено честно: недостающий бит выдаётся **временно**, только на время проверки, и сразу возвращается обратно; факт и причина фиксируются в журнале.

Правило 3. Каталог перекрывает всё, что внутри

Доступ проверяется на всём пути к файлу. Если на каталог нет права `x`, содержимое недостижимо, какими бы щедрыми ни были права самих файлов.

В прогоне это видно по каталогу `pzs11` с режимом `0700`: в его таблице у всех субъектов кроме `iit11` и `root` стоят прочерки во всех 25 файлах — при том что некоторые из этих файлов имеют режим `0777`.

Правило 4. Администратор — особый случай, но не всемогущий

Пользователь `root` обходит проверку битов при чтении и записи: это возможность `CAP_DAC_OVERRIDE`. Но для **выполнения** сделано исключение: `root` может запустить файл только если хотя бы один бит `x` где-нибудь выставлен.

Поэтому в таблице прогона у `root` напротив `file11 (0400)` стоит `rw-`, а напротив `file14 (0700)` — `rwX`. Разница именно в этом.

Дополнительно: кто может завершить процесс

Право на файл и право на запущенный из него процесс — разные вещи. Сигнал процессу может послать только его владелец или `root`. Права на исполняемый файл здесь роли не играют совсем.

Отдельный столбец в журнале — `sudo:iit21`. Пользователь `iit21` по заданию получает административные привилегии, и через `sudo` он завершает любой процесс. Это показывает разницу между **своими** правами субъекта и **делегированными** через механизм повышения привилегий.

4. Что реализовано в части 1.1

Программа `acl_lab.py` выполняет все 17 шагов задания последовательно и печатает результат таблицами в журнал `part1.log`.

Шаги	Что делается	Как в программе
1-5	создание субъектов	группы <code>group_iit1</code> , <code>group_iit2</code> ; пользователи <code>iit11</code> , <code>iit12</code> в первую, <code>iit21</code> , <code>iit22</code> во вторую, <code>iit3</code> без группы; <code>iit21</code> получает административные привилегии
6-11	создание каталогов	<code>pzs11</code> (0700), <code>pzs12</code> (0070), <code>pzs13</code> (0007), <code>pzs14</code> (0777), <code>pzs15</code> (0700, владелец <code>root</code>)
12-13	создание 25 файлов в каждом из четырёх каталогов	пять групп по пять файлов: для владельца, группы, остальных, всех и администратора; файлы <code>filex5</code> содержат блокирующий <code>read</code> , остальные — <code>echo</code>
14	проверка чтения, записи и выполнения	каждый из 100 файлов проверяется шестью субъектами: <code>iit11</code> , <code>iit12</code> , <code>iit21</code> , <code>iit22</code> , <code>iit3</code> , <code>root</code>
15	запуск процессов и попытки их завершить	сначала запуск строго по заданию (от <code>iit11</code>), затем демонстрация правил: каждый файл запускается тем субъектом, которому его класс разрешает выполнение
16	проверка операций с каталогами	листинг, создание нового файла, удаление каждого существующего — для всех субъектов
17	полная очистка	удаление файлов, каталогов, пользователей и групп

Особенности реализации, которые стоит упомянуть на защите

- **Эталонная копия файлов** лежит в `/var/tmp/pzs_template`, то есть вне `/pzs`. Иначе она попадала бы в проверки листинга каталогов на шаге 16 и искажала результат.
- **Восстановление состояния.** Проверки записи и удаления разрушают набор файлов, поэтому после каждой такой проверки файл восстанавливается из эталона, а режимы каталога возвращаются к исходным.
- **Защита от запуска не в контейнере.** Программа создаёт и удаляет системных пользователей, поэтому перед работой проверяет, что окружение одноразовое.
- **Временная выдача битов** на шаге 15 выполняется явно, с записью в журнал, и отменяется сразу после проверки.

5. Неочевидные результаты прогона

Эти четыре результата выглядят как ошибки, а на самом деле подтверждают правильность модели. Именно их и стоит объяснять.

Наблюдение	Объяснение
Ни один файл <code>filex5</code> не запускается пользователем <code>iit11</code> — 20 отказов подряд	Правило 2: сценарию нужен бит чтения, а по заданию у этих файлов только бит выполнения. Плюс правило 1: <code>iit11</code> — владелец, значит для <code>file25</code> (права группе) он всё равно попадает в класс владельца с нулевыми правами

В каталоге <code>pzs11</code> прочерки у всех, кроме <code>iit11</code> и <code>root</code> , даже у файлов <code>0777</code>	Правило 3: каталог имеет режим <code>0700</code> , войти в него больше никому
У <code>root</code> напротив <code>file11</code> стоит <code>rw-</code> , а не <code>rwX</code>	Правило 4: администратор обходит проверку чтения и записи, но для выполнения нужен хотя бы один бит <code>x</code> , а у <code>0400</code> его нет
В каталоге <code>pzs13</code> (права «остальным») удалять файлы могут все пятеро пользователей	Право удаления берётся из прав на каталог. Режим <code>0007</code> даёт классу «остальные» право записи в каталог, а владелец каталога — <code>root</code> , поэтому в класс «остальные» попадают все пользователи

6. Часть 1.2: контроль доступа в СУБД

6.1. Зачем отдельная часть про базу данных

Файловая система защищает файлы. Но данные приложения лежат не в файлах, а внутри СУБД, и у неё **собственный** механизм контроля доступа — операционная система про таблицы ничего не знает. Права на каталог PostgreSQL не помогут, если внутри базы всем всё разрешено.

Вторая мысль части 1.2 — **принцип минимальных привилегий**: пользователь программного продукта должен работать с минимально возможным уровнем прав. Нарушение этого правила приводит к тому, что при сбое или атаке злоумышленник получает не ограниченный, а полный доступ.

6.2. Как устроены права в PostgreSQL

Понятие	Что это простыми словами
Роль	и пользователь, и группа одновременно. Роль с правом входа — это учётная запись; роль без права входа (NOLOGIN) удобна как группа
Членство в роли	<code>GRANT user_role TO user_sub</code> — учётная запись получает привилегии роли, как участник группы
Привилегия	конкретное разрешённое действие: <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>CONNECT</code> , <code>USAGE</code>
Атрибут роли	особые полномочия самой роли: <code>SUPERUSER</code> , <code>CREATEDB</code> , <code>CREATEROLE</code> , <code>BYPASSRLS</code> . Не наследуются через членство — только выдаются напрямую
PUBLIC	псевдороль «все роли кластера». Многие права достаются ей по умолчанию, поэтому их нужно явно отзывать
Постолбцовый грант	разрешение читать не всю таблицу, а перечисленные столбцы: <code>GRANT SELECT (id, data)</code>
Построчная защита (RLS)	политика, ограничивающая какие строки видны роли. Ограничение по столбцам и по строкам — два разных измерения

ДВЕ ЛОВУШКИ, КОТОРЫЕ НАДО ЗНАТЬ

Первая: PUBLIC. Без явного `REVOKE ALL ON DATABASE ... FROM PUBLIC` право подключения достаётся вообще всем учётным записям кластера. То же со схемой `public` и со служебной базой `postgres` — через неё пользователь и гость видели бы каталоги всего кластера.

Вторая: владелец таблицы не подчиняется RLS. Если владельцем таблицы окажется та же роль, под которой работает приложение, все политики построчной защиты превратятся в декорацию.

6.3. Ролевая модель, которую требует задание

Роль	Требование задания	Как выражено в привилегиях
------	--------------------	----------------------------

Администратор	полный доступ ко всем подсистемам, максимальные привилегии	ALL PRIVILEGES на схему и таблицы WITH GRANT OPTION ; атрибуты CREATEDB , CREATEROLE , BYPASSRLS напрямую учётной записи
Пользователь	полные либо частичные права в выданной подсистеме	USAGE на схему и SELECT/INSERT/UPDATE/DELETE на таблицы, но без DDL : менять структуру нельзя
Гость	только чтение отдельных фрагментов	ограничение сразу по двум измерениям: постолбцовый SELECT (id, data) плюс политика RLS, показывающая только строки с visibility = 'public'

7. Что реализовано в части 1.2

Программа `pg_acl.py` объединяет четыре требуемые заданием программы в одно меню:

Пункт	Назначение	Что делает
1	установка СУБД	установка PostgreSQL, запуск кластера, настройка <code>pg_hba.conf</code>
2	настройка политики	создание трёх ролей и трёх учётных записей, базы <code>app_db</code> , схемы <code>app</code> , таблиц <code>public_data</code> и <code>private_data</code> , выдача привилегий, отзыв прав у <code>PUBLIC</code> , включение RLS
3	проверка корректности	22 проверки: для каждой указано ожидание (успех или отказ) и фактический результат
4	полное удаление	удаление пакетов, каталога данных и системного пользователя <code>postgres</code>

Результат проверок: 22 из 22 PASS

Проверки построены так, что проверяют не только разрешения, но и **запреты** — это важнее. Примеры из журнала `part2.log`:

Проверка	Ожидание	Смысл
Пользователь: создание таблицы	отказ	у роли нет DDL
Пользователь: смена владельца таблицы	отказ	нельзя обойти ограничение через смену владельца
Гость: чтение всех столбцов <code>public_data</code>	отказ	работает постолбцовый грант
Гость: чтение <code>private_data</code>	отказ	таблица гостю не выдана вовсе
Гость: подключение к базе <code>postgres</code>	отказ	право у <code>PUBLIC</code> отозвано
Пользователь: расширение прав гостя не имеет эффекта	отказ	не-владелец не может выдать привилегию, которой не владеет
Гость: построчная защита <code>public_data</code>	3 из 5	RLS скрыл две внутренние записи из пяти

ТОНКОСТЬ, КОТОРУЮ СТОИТ ЗНАТЬ

PostgreSQL на команду `GRANT` от лица не-владельца выдаёт не ошибку, а предупреждение. Поэтому проверка «расширение прав гостя не имеет эффекта» устроена иначе: программа выполняет `GRANT` от имени пользователя, а затем проверяет, появился ли у гостя доступ фактически. Проверять код возврата здесь было бы недостаточно.

8. Вопросы и ответы

В. Что такое ACL и почему права UNIX — это тоже ACL?

О. ACL — таблица записей «субъект → разрешённые действия над объектом». В UNIX такая таблица сокращена до трёх обязательных записей: владелец, группа-владелец, остальные. Это вырожденный, но полноценный ACL: каждый из трёх классов представлен отдельной записью ACE.

В. В каком порядке проверяются записи ACL?

О. Владелец → именованный пользователь → группа → остальные. Выбирается запись, наиболее точно соответствующая субъекту, и доступ определяется **только ею**. Если выбранной записи прав не хватает, будет отказ, даже когда другая запись их разрешает.

В. Почему владелец файла не смог его прочитать?

О. Потому что у файла режим `0040`: права выданы классу «группа», а владелец попадает в класс «владелец» с правами `---`. Правило «решает одна запись» не позволяет добрать права из другого класса. Это ожидаемое поведение POSIX.

В. Можно ли запретить владельцу менять права своего файла?

О. Нет. Создатель объекта является его начальным владельцем, и ограничить владельца от изменения разрешений нельзя — он в любой момент вернёт себе доступ командой `chmod`. Именно поэтому в программе владельцем каталогов, которые должны быть закрыты от `ititll`, назначен `root`.

В. Почему для запуска сценария не хватает бита выполнения?

О. Сценарий исполняет интерпретатор, который работает от имени запускающего пользователя и должен сначала **прочитать** файл. Значит нужны оба бита: `x` и `r`. Для скомпилированной программы достаточно одного `x`, потому что её загружает ядро.

В. От чего зависит право удалить файл?

О. От права записи на **каталог**, а не на файл. Имея `w` на каталог, можно удалить любой файл внутри, включая чужой с режимом `0000`. Обратное тоже верно: полные права на файл без `w` на каталог удалить его не позволяют.

В. Что даёт бит `x` на каталоге?

О. Право войти в каталог и обратиться к файлу по имени. Без него содержимое недостижимо, даже если известны имена. Бит `r` на каталоге — это отдельное право получить список имён; можно иметь `x` без `r`: тогда файл открывается по точному имени, но список не показывается.

В. Всё ли может `root`?

О. При чтении и записи он обходит проверку битов — это возможность `CAP_DAC_OVERRIDE`. Но выполнить файл он может только если хотя бы один бит `x` выставлен. Поэтому у `root` для режима `0400` получается `rw-`, а для `0700` — `rwx`.

В. Кто может завершить чужой процесс?

О. Только владелец процесса и `root`. Права на исполняемый файл значения не имеют. Пользователь с административными привилегиями (`iit21` по заданию) делает это через `sudo` — то есть не своими правами, а делегированными.

В. Что такое ACL по умолчанию?

О. Это ACL каталога, который наследуется создаваемыми в нём объектами. Унаследованные права затем ограничиваются параметром режима доступа из системного вызова создания: итоговое право — **пересечение** наследованного и запрошенного.

В. Чем роль в PostgreSQL отличается от пользователя?

О. Это одно и то же: роль с правом входа работает как учётная запись, роль без права входа удобна как группа. В работе три роли-группы (`admin_role`, `user_role`, `guest_role`) и три учётных записи, включённые в них.

В. Почему административные атрибуты выданы учётной записи напрямую, а не через роль?

О. Потому что атрибуты роли — `SUPERUSER`, `CREATEDB`, `CREATEROLE`, `BYPASSRLS` — **не наследуются** через членство, в отличие от привилегий на объекты. Их нужно выдавать той роли, которая будет входить в систему.

В. Что такое PUBLIC и зачем у неё отзывать права?

О. `PUBLIC` — псевдороль «все роли кластера». Право подключения к базе и права на схему `public` достаются ей по умолчанию, поэтому без явного `REVOKE` ограничения не работают: любая учётная запись подключится к базе и увидит служебные каталоги через базу `postgres`.

В. Как гостю выдано «только чтение отдельных фрагментов»?

О. Двумя разными механизмами сразу. По столбцам — постолбцовый грант `SELECT (id, data)`: служебный столбец `visibility` гостю недоступен. По строкам — политика RLS, показывающая только записи с `visibility = 'public'`. В проверке видно результат: 3 строки из 5.

В. Что такое принцип минимальных привилегий и где он применён?

О. Субъект получает ровно те права, без которых не может выполнить задачу. В работе: пользователь получил операции с данными, но не DDL; гость — чтение двух столбцов и трёх строк; право подключения отозвано у `PUBLIC` и выдано адресно. Смысл в том, что при сбое или атаке ущерб ограничен выданными правами.

В. Почему обе программы запускаются только в контейнере?

О. Они выполняют разрушительные операции: создают и удаляют системных пользователей и группы, а пункт 4 второй программы безвозвратно удаляет пакеты PostgreSQL, каталог данных и системного пользователя `postgres`. Одноразовый контейнер гарантирует, что после прогона на машине ничего не останется.

9. Итог

Семантическая составляющая. Работа показывает, что контроль доступа — это не набор запретов, а модель с точными правилами вычисления решения. Одна и та же тройка «субъект — объект — право» реализуется в файловой системе тремя классами с приоритетом владельца, а в СУБД — ролями, привилегиями и политиками. Большинство неожиданных результатов объясняются не ошибками, а строгостью модели: решение принимает ровно одна запись ACL, права на каталог и на файл означают разное, а право удаления вообще определяется каталогом.

Прагматическая составляющая. Получены две программы, которые настраивают разграничение доступа и, что важнее, **проверяют** его таблично: 100 файлов × 6 субъектов в первой части и 22 проверки разрешений и запретов во второй. Такой формат превращает утверждение «права настроены правильно» в измеримый результат. Настроенная ролевая модель PostgreSQL переиспользуется во второй лабораторной работе как готовая политика безопасности СУБД.